UTS ALGORITMA PEMROGRAMAN LANJUT

PEMBUATAN APLIKASI ALGOLEARN BERBASIS C++ UNTUK PEMBELAJARAN DASAR ALGORITMA PEMROGRAMAN DAN STRUKTUR DATA

Disusun oleh:

Kelas A'25

KELOMPOK 6

Muhammad Alfauzi Syahputra	**(2509106006)**
Syarafina Nur Amalia	**(2509106016)**
Diftya Azzahra	**(2509106042)**

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK UNIVERSITAS MULAWARMAN

SAMARINDA

2026

KATA PENGANTAR

Puji syukur kami haturkan ke hadirat Allah SWT yang telah melimpahkan Rahmat dan hidayah-Nya, sehingga tim penulis bisa menyelesaikan karya tulis ilmiah ini yang berjudul “Pembuatan Aplikasi Algolearn Berbasis C++ Untuk Pembelajaran Dasar Algoritma Pemrograman dan Struktur Data." Tidak lupa juga kami mengucapkan terima kasih kepada semua pihak yang telah memberikan kontribusi dalam penyusunan karya ilmiah ini.

Sebagai tim penulis, kami menyadari bahwa masih terdapat kekurangan, baik dari penyusunan maupun tata bahasa penyampaian dalam karya ilmiah ini. Oleh karena itu, tim penulis dengan rendah hati menerima saran dan kritik dari pembaca. Tim penulis berharap semoga karya ilmiah yang tim penulis susun ini memberikan manfaat dan inspirasi bagi pembaca.

Samarinda, 28 Maret 2026

DAFTAR ISI

KATA PENGANTAR **2**
DAFTAR ISI **3**
BAB I **6**
PENDAHULUAN **6**
1.1 Latar Belakang 6
1.2 Rumusan Masalah 7
1.3 Tujuan 7
1.4 Manfaat 7
1.4.1 Teoretis 7
1.4.2 Praktis 7
BAB II **8**
TINJAUAN PUSTAKA **8**
2.1 Algoritma 8
2.2 Bahasa Pemrograman C++ 8
2.3 Array 8
2.4 Pointer 8
2.5 Struct 9
2.6 Fungsi dan Prosedur 9
2.7 Sorting 9
2.8 Searching 10
BAB III **11**
METODE PENELITIAN **11**
3.2 Teknik Pengumpulan Data 12
3.3 Perancangan Proses Algoritma *Sorting* 12
3.4 Perancangan Proses Algoritma *Searching* 13
3.5 Perancangan Program/Aplikasi 13
BAB IV **16**
HASIL DAN PEMBAHASAN **16**
4.1 Array & Array Multi Dimensi 16
4.1.1 String 16
4.1.2 Array Statis 16

4.1.3 Array Dinamis 17
4.1.4 Array 1 Dimensi 17
4.1.5 Array Multi Dimensi (2 Dimensi) 18
4.1.6 Array Multi Dimensi (3 Dimensi) 18
4.1.7 Penjumlahan Matriks 19
4.1.8 Pengurangan Matriks 19
4.1.9 Perkalian Matriks 20
4.1.10 Transpose Matriks 21
4.1.11 Invers Matriks 21
4.2 Pointer & Struct 22
4.2.1 Pointer 22
4.2.2 Struct 22
4.2.3 Array & Struct 23
4.2.4 Pointer & Struct 23
4.2.5 Union 24
4.3 Prosedur & Fungsi 24
4.3.1 Prosedur 24
4.3.2 Fungsi 25
4.3.3 Parameter 25
4.3.4. Variabel Lokal 26
4.3.5 Variabel Global 26
4.3.6 Variabel *Static* 27
4.3.4 Fungsi Rekursif 27
4.4 *Sorting* 28
4.4.1 Insertion Sort 28
4.4.2 Heap Sort 29
4.4.3 Merge Sort 30
4.4.4 Quick Sort 31
4.4.5 Tree Sort 32
4.4.6 Exchange Sort 33
4.4.7 Radix Sort 33
4.4.8 Shell Sort 34
4.4.9 Bubble Sort 35

4.4.10 Selection Sort 35
4.5 Searching 36
4.5.1 Fibonacci Search 36
4.5.2 Binary Search 37
4.5.3 Interpolation Search 38
4.5.4 Sequential Search 40
BAB V 41
KESIMPULAN DAN SARAN 41
5.1 Kesimpulan 41
5.2 Saran 41
DAFTAR PUSTAKA 42

BAB I

PENDAHULUAN

1.1 Latar Belakang

Algoritma adalah sebuah logika, metode, dan tahapan yang disusun secara sistematis untuk menyelesaikan suatu masalah. Dengan kata lain, algoritma merupakan serangkaian langkah sistematis atau prosedur yang dijabarkan untuk menyelesaikan suatu permasalahan atau menjalankan tugas tertentu berdasarkan instruksi yang jelas dan logis. Algoritma dan struktur data merupakan bagian penting dalam pembelajaran pemrograman, khususnya pada tahap lanjutan, karena berperan dalam pengembangan kemampuan pemecahan masalah dan *algorithmic thinking*. Bidang ilmu komputer dan rekayasa perangkat lunak menuntut pemahaman yang mendalam tentang struktur data dan algoritma. Konsep-konsep ini menjadi landasan dalam memecahkan masalah yang kompleks dan sangat diperlukan untuk mengembangkan sistem perangkat lunak yang efisien dan dapat diskalakan (Talumewo et al, 2025).

Namun, memahami algoritma dan struktur data pemrograman merupakan tantangan yang cukup besar bagi mahasiswa program sarjana ilmu komputer dan rekayasa perangkat lunak (Kitchin, 2017). Pembelajaran pada materi ini sering dianggap sulit oleh mahasiswa, terutama apabila penyampaiannya lebih berfokus pada teori dibandingkan praktik, sehingga menyulitkan pemahaman konsep yang bersifat abstrak dan kompleks (Mtaho et al, 2024). Namun penelitian telah menunjukkan bahwa visualisasi dan animasi dapat secara efektif meningkatkan hasil belajar mahasiswa dalam mata kuliah pemrograman dan struktur data. Dilakukan penelitian yaitu dengan penggunaan teknologi visualisasi dalam pendidikan jarak jauh dan menemukan bahwa visualisasi meningkatkan kualitas dan efektivitas pelatihan, serta alat visualisasi juga membantu meningkatkan pembelajaran jarak jauh bagi para siswa. Namun, mereka mengakui perlunya penelitian lebih lanjut untuk mengeksplorasi aspek-aspek lain dari visualisasi dalam pelatihan profesional (Andreev et al, 2021) .

Perháč dan Šimoňák (2022) juga mengembangkan sistem visualisasi algoritma baru yang dirancang untuk mengajarkan pemrograman struktur data dan algoritma. Studi mereka bertujuan untuk meningkatkan kegunaan, kemampuan perluasan, ketersediaan, dan fungsionalitas sistem yang ada. Sistem yang baru dikembangkan menyediakan sebagian besar fungsionalitas untuk visualisasi struktur data dan dinilai mudah dipahami serta bermanfaat dalam pembelajaran. Para responden juga meminta penambahan visualisasi dan fungsionalitas baru, yang menunjukkan potensi pengembangan lebih lanjut di bidang ini.

1.2 Rumusan Masalah

1. Bagaimana perancangan struktur menu dan pembagian materi pada aplikasi AlgoLearn sehingga topik algoritma dan struktur data dapat disajikan secara sistematis dalam bentuk aplikasi berbasis *Command Line Interface* (CLI)?
2. Bagaimana implementasi materi Array & Array Multi Dimensi, Pointer & Struct, Prosedur & Fungsi, Sorting, dan Searching ke dalam aplikasi AlgoLearn menggunakan bahasa C++?

1.3 Tujuan

Mengembangkan media pembelajaran algoritma dan struktur data berbasis C++ yang dapat digunakan sebagai sarana bantu belajar bagi mahasiswa. Aplikasi ini dirancang untuk menyajikan materi secara terstruktur, dilengkapi dengan contoh implementasi program serta simulasi sederhana, sehingga dapat membantu pengguna memahami konsep-konsep dasar seperti array, pointer, fungsi, sorting, dan searching dengan lebih sistematis dan aplikatif.

1.4 Manfaat

1.4.1 Teoretis

Secara teoritis, pengembangan aplikasi AlgoLearn diharapkan dapat memberikan kontribusi dalam bentuk referensi sederhana mengenai penerapan konsep algoritma dan struktur data ke dalam media pembelajaran berbasis perangkat lunak. Selain itu, aplikasi ini diharapkan dapat menambah wawasan mengenai bagaimana materi algoritma dan struktur data dapat disajikan secara lebih sistematis melalui pendekatan aplikasi berbasis command line interface.

1.4.2 Praktis

Secara praktis, aplikasi ini dapat dimanfaatkan sebagai media pembelajaran alternatif yang membantu mahasiswa memahami materi secara lebih terstruktur melalui penyajian konsep, contoh implementasi, dan simulasi sederhana. Selain itu, aplikasi ini juga dapat mendukung proses belajar mandiri mahasiswa dalam memperdalam pemahaman terhadap materi yang berkaitan dengan algoritma dan struktur data.

BAB II

TINJAUAN PUSTAKA

2.1 Algoritma

Algoritma adalah langkah berurutan dan tertulis untuk memecahkan masalah. Dan algoritma pemrograman adalah langkah-langkah yang ditulis secara berurutan untuk memecahkan suatu masalah. Sedangkan program jika dipahami dalam konteks teknologi informasi adalah aplikasi yang digunakan untuk melakukan tugas atau masalah, seperti aplikasi untuk mengerjakan dokumen, desain grafis, perhitungan dan aplikasi lainnya (Khesya Nurhaliza, 2021).

2.2 Bahasa Pemrograman C++

Bahasa Pemrograman C++ adalah bahasa pemrograman tingkat tinggi yang biasa digunakan untuk pengembangan perangkat lunak, mulai dari aplikasi dekstop hingga permainan di komputer dan sistem operasi. C++ adalah bahasa pemrograman yang cukup sulit tetapi menawarkan fleksibilitas yang lebih besar dalam mengembangkan aplikasi yang kompleks. Beberapa fitur utama C++ adalah kemampuan untuk mengorganisir kode dalam objek, kemampuan untuk menggunakan pointer, penggunaan kelas untuk membuat objek, dan kemampuan untuk melakukan *overloading* operator. Selain itu C++ juga mendukung konsep pemrograman generik, yaitu kemampuan untuk menulis kode yang dapat digunakan untuk membuat berbaagai jenis data. Bahasa pemrograman ini bisa dipakai di berbagai platform, termasuk Windows, Mac OS, dan Linux. (Ritonga Adawiyah & Yahfizham, 2023).

2.3 Array

Array adalah suatu struktur yang terdiri dari sejumlah elemen yang memiliki tipe data yang sama. Elemen-elemen array tersusun secara sekuensial dalam memori komputer. Semua elemem array bertipe sama. Array cocok untuk organisasi kumpulan data homogen yang ukuran atau jumlah elemen maksimumnya telah diketahui dari awal. Homogen adalah bahwa setiap elemen dari sebuah array tertentu haruslah mempunyai tipe data yang sama. Array dapat dibedakan menjadi beberapa jenis, seperti array satu dimensi, array dua dimensi, dan array multidimensi (Pratama Aldi, 2020).

2.4 Pointer

Pointer adalah variabel yang menyimpan alamat memori dari variabel lain. Anggap saja sebagai "penunjuk" yang mengarahkan kita ke lokasi di memori tempat data yang kita inginkan

berada. Menggunakan pointer memungkinkan kita untuk mengakses dan memanipulasi data secara tidak langsung, serta memungkinkan kita membangun struktur data yang dinamis dan melakukan manajemen memori yang efisien (Ginting Nanda Hafiz Subhan et al, 2023).

2.5 Struct

Struct atau struktur adalah tipe data yang dapat digunakan untuk menyimpan data yang berbeda dalam satu variabel. *Struct* sering digunakan untuk menyimpan data yang terkait, seperti data mahasiswa atau data produk (Lestanto Yusuf , 2023).

2.6 Fungsi dan Prosedur

Fungsi (*function*) adalah suatu kumpulan instruksi/perintah yang dikelompokkan menjadi satu berupa blok pada kode program, letaknya terpisah dari program utama atau program yang menggunakan fungsi tersebut. Fungsi memiliki nama tertentu yang bersifat unik dan digunakan untuk mengerjakan suatu tujuan tertentu. Dalam bahasa pemrograman lain fungsi dapat disebut sebagai subrutin (basic, VB). Fungsi hanya akan berjalan ketika nama dari fungsi tersebut dipanggil. Sebuah fungsi dapat menerima data melalui parameter fungsi. Fungsi digunakan untuk melakukan beberapa aksi dan memberikan kemudahan dalam penggunaan kembali kode yang ingin dieksekusi secara berulang sehingga sebuah kode dapat ditulis satu kali dan digunakan berkali-kali.

Selain itu, jika dilihat dari nilai yang dikembalikan oleh fungsi kepada pemanggilnya, ada fungsi yang tidak mengembalikan nilai (void) atau bisa disebut sebagai prosedur. Fungsi yang tidak mengembalikan nilai sering juga disebut sebagai prosedur. Ciri dari fungsi yang tidak mengembalikan nilai adalah: (1) Tidak terdapat keyword (kata kunci) return, (2) Tidak terdapat tipe data dalam deklarasi fungsi, (3) Menggunakan keyword void, (4) Tidak memiliki *return value* (Hanief Shofwan & Jepriana Wayan I, 2020).

2.7 Sorting

Algoritma pengurutan (sorting) adalah proses menyusun elemen data dalam urutan tertentu, yang menjadi dasar bagi banyak operasi dalam ilmu komputer, seperti pencarian dan analisis data. Secara umum, algoritma sorting dapat dibagi menjadi dua kategori, yaitu algoritma berbasis perbandingan dan non-perbandingan.

Algoritma berbasis perbandingan, seperti *Heap Sort, Quick Sort, Merge Sort*, dan *Shell Sort,* bekerja dengan membandingkan elemen secara berpasangan. Algoritma ini memiliki batas bawah kompleksitas waktu sebesar O(n log n). Keunggulan dari algoritma ini adalah fleksibilitasnya dalam menangani berbagai tipe data, meskipun efisiensinya sangat bergantung

pada jumlah perbandingan yang dilakukan. Sementara itu, algoritma non-perbandingan, seperti *Counting Sort*, memanfaatkan informasi khusus tentang data, seperti rentang nilai, untuk melakukan pengurutan tanpa perbandingan langsung. Pendekatan ini memungkinkan tercapainya kompleksitas waktu yang lebih rendah dalam kondisi tertentu (Pujiono Prayogo Imam et al, 2025).

2.8 Searching

Searching atau pencarian merupakan salah satu konsep dasar dalam ilmu komputer yang berperan penting dalam berbagai aspek pemrosesan data dan pengembangan perangkat lunak. Secara umum, searching merujuk pada proses sistematis untuk menemukan elemen tertentu dalam suatu kumpulan data atau struktur data. Elemen yang dicari bisa berupa angka, karakter, string, atau bahkan objek kompleks, tergantung pada konteks dan kebutuhan aplikasi. Dalam pemrograman, terdapat beberapa jenis algoritma searching yang digunakan secara luas, masing-masing dengan kompleksitas dan efisiensi yang berbeda. Penerapan algoritma pencarian menjadi sangat krusial karena hampir seluruh proses dalam sistem komputasi modern-mulai dari pencarian kata dalam dokumen, pencocokan nama pengguna di basis data, hingga pengambilan informasi dalam sistem kecerdasan buatan melibatkan bentuk tertentu dari operasi pencarian. Oleh karena itu, efisiensi algoritma searching sangat menentukan kinerja keseluruhan dari suatu program atau sistem, terutama ketika harus menangani data dalam skala besar dan waktu yang terbatas (Kusnadi Yahdi, 2025).

BAB III

METODE PENELITIAN

3.1 Tahapan Pembuatan Aplikasi

Tahapan pembuatan aplikasi AlgoLearn dilakukan melalui beberapa langkah sistematis agar aplikasi dapat dikembangkan dengan baik dan sesuai dengan tujuan pembelajaran. Adapun tahapan yang dilakukan adalah sebagai berikut:

1. Analisis Kebutuhan

   Pada tahap ini dilakukan identifikasi kebutuhan aplikasi, baik dari sisi materi maupun fitur. Aplikasi dirancang untuk membantu pengguna memahami konsep dasar algoritma dan struktur data, seperti array, pointer, fungsi, *sorting*, dan *searching*.
2. Perancangan Sistem

   Setelah kebutuhan ditentukan, dilakukan perancangan sistem menggunakan pendekatan modular. Aplikasi dirancang berbasis *Command Line Interface* (CLI) dengan struktur menu utama dan submenu untuk setiap topik materi. Perancangan ini bertujuan agar program memiliki struktur yang rapi, mudah dipahami, dan mudah dikembangkan.
3. Implementasi Program

   Pada tahap ini dilakukan proses pengkodean menggunakan bahasa C++. Program dibagi ke dalam beberapa file sesuai dengan topik materi. File *main.cpp* berfungsi sebagai pusat kontrol program, sedangkan file lainnya digunakan untuk mengelola masing-masing materi, seperti Array, Pointer dan Struct, Prosedur dan Fungsi, *Sorting*, dan *Searching*.
4. Pengujian Program

   Setelah program selesai dibuat, dilakukan pengujian untuk memastikan setiap fitur berjalan dengan baik. Pengujian dilakukan dengan mencoba setiap menu dan memastikan output yang dihasilkan sesuai dengan yang diharapkan.

3.2 Teknik Pengumpulan Data

Metode pengumpulan data dalam penelitian ini dilakukan dengan menggunakan studi pustaka. Data yang digunakan merupakan data sekunder, yaitu data yang diperoleh dari berbagai sumber tertulis seperti buku, jurnal ilmiah, artikel dan modul pembelajaran. Pengumpulan data dilakukan untuk memperoleh landasan teori, materi pembelajaran, serta informasi teknis yang mendukung proses perancangan dan pembuatan aplikasi.

Data yang dikumpulkan meliputi konsep-konsep dasar algoritma dan struktur data, seperti array, pointer, struct, procedure, fungsi, sorting, dan searching. Selain itu, dikumpulkan pula referensi yang membahas kesulitan mahasiswa dalam mempelajari algoritma dan struktur data serta pendekatan pembelajaran yang dapat membantu meningkatkan pemahaman konsep. Hasil pengumpulan data tersebut kemudian dianalisis dan disesuaikan dengan kebutuhan aplikasi sehingga materi yang disajikan tetap relevan dengan tujuan pembelajaran.

3.3 Perancangan Proses Algoritma *Sorting*

Proses *sorting* dirancang untuk mengurutkan sekumpulan data numerik dari nilai terkecil ke terbesar. Dalam implementasinya pada AlgoLearn, data input disimpan dalam array atau vector, kemudian diproses menggunakan algoritma pengurutan tertentu. Aplikasi ini memuat beberapa algoritma sorting, yaitu Insertion Sort, Heap Sort, Merge Sort, Quick Sort, Tree Sort, Exchange Sort, Radix Sort, Shell Sort, Bubble Sort, dan Selection Sort.

Secara teknis, input pada proses *sorting* adalah sekumpulan data yang belum terurut. Proses dilakukan dengan menerapkan metode pengurutan sesuai algoritma yang dipilih pengguna. Setiap algoritma bekerja dengan mekanisme yang berbeda, misalnya penyisipan elemen, pertukaran elemen, pembagian data ke subbagian, atau pengelompokan berdasarkan nilai digit. Output dari proses ini adalah data yang telah terurut serta tampilan tahapan atau iterasi yang menunjukkan perubahan susunan data selama proses berlangsung.

Sebagai contoh, pada Insertion Sort, proses dilakukan dengan mengambil elemen mulai dari indeks kedua, lalu membandingkannya dengan elemen-elemen sebelumnya hingga menemukan posisi yang tepat. Pada Bubble Sort, proses dilakukan dengan membandingkan dua elemen yang berdekatan dan menukarnya apabila urutannya salah. Sementara itu, pada Merge Sort dan Quick Sort, pengurutan dilakukan menggunakan pendekatan divide and conquer, yaitu membagi data ke bagian-bagian kecil, memprosesnya, lalu menggabungkan kembali hingga urutan data menjadi benar.

3.4 Perancangan Proses Algoritma *Searching*

Proses *searching* dirancang untuk mencari suatu nilai tertentu di dalam sekumpulan data. Dalam aplikasi AlgoLearn, proses *searching* digunakan untuk membantu pengguna memahami bagaimana suatu elemen dicari berdasarkan algoritma pencarian yang berbeda. Aplikasi ini memuat beberapa algoritma *searching*, yaitu Fibonacci Search, Binary Search, Interpolation Search, dan Sequential Search.

Secara teknis, input pada proses searching terdiri atas dua komponen, yaitu sekumpulan data dan nilai target yang ingin dicari. Proses dilakukan dengan menjalankan metode pencarian tertentu untuk menentukan apakah target tersebut ada di dalam data dan pada posisi berapa target ditemukan. Output dari proses ini adalah indeks atau posisi data yang ditemukan, atau informasi bahwa data tidak ditemukan.

Pada Sequential Search, proses pencarian dimulai dari elemen pertama dan dilakukan satu per satu hingga target ditemukan atau hingga elemen terakhir selesai diperiksa. Pada Binary Search, data harus dalam keadaan terurut. Proses dilakukan dengan membandingkan target terhadap elemen tengah, kemudian ruang pencarian dipersempit ke bagian kiri atau kanan sesuai hasil perbandingan. Pada Interpolation Search, posisi pencarian diperkirakan berdasarkan distribusi nilai data, sehingga lebih efektif pada data yang penyebarannya relatif merata. Sementara pada Fibonacci Search, pencarian dilakukan dengan memanfaatkan bilangan Fibonacci untuk menentukan batas-batas pembagian area pencarian.

Dalam implementasinya, aplikasi menampilkan data contoh, meminta pengguna memasukkan nilai target, lalu menjalankan algoritma pencarian sesuai pilihan submenu. Selama proses berlangsung, aplikasi menampilkan langkah pencarian secara bertahap agar pengguna memahami bagaimana target dibandingkan dengan elemen-elemen pada array hingga target ditemukan atau dinyatakan tidak ada.

3.5 Perancangan Program/Aplikasi

Perancangan program pada aplikasi AlgoLearn dilakukan dengan pendekatan modular, yaitu memisahkan setiap bagian program berdasarkan fungsi dan topik materi yang dibahas. Pendekatan ini dipilih agar struktur program lebih rapi, mudah dipelihara, dan mudah dikembangkan pada tahap berikutnya. Selain itu, dengan pembagian modul yang jelas, setiap materi dapat dikelola secara terpisah tanpa mengganggu bagian program yang lain.

Aplikasi dirancang berbasis Command Line Interface (CLI), sehingga interaksi antara pengguna dan sistem dilakukan melalui tampilan teks di terminal. Pengguna mengakses materi

dengan memilih menu menggunakan input angka. Setiap angka mewakili topik tertentu, sehingga mekanisme navigasi aplikasi menjadi sederhana dan mudah dipahami. Menu utama aplikasi dirancang sebagai pusat navigasi yang menghubungkan seluruh submenu materi. Struktur menu utama pada aplikasi terdiri atas enam pilihan, yaitu:

1. Array & Array Multi Dimensi
2. Pointer & Struct
3. Procedure & Fungsi
4. Sorting
5. Searching
6. Keluar

Berdasarkan struktur tersebut, alur program dimulai dengan menampilkan menu utama kepada pengguna. Setelah itu, pengguna memasukkan angka pilihan menu. Sistem kemudian membaca input dan melakukan proses untuk menentukan submenu yang akan dijalankan. Jika pengguna memilih salah satu topik materi, sistem akan menampilkan submenu sesuai kategori tersebut. Jika pengguna memilih angka 0, program akan dihentikan. Apabila input tidak sesuai dengan pilihan yang tersedia, sistem akan menampilkan pesan kesalahan dan mengarahkan pengguna kembali ke menu utama.

Secara teknis, alur kerja menu utama menggunakan kombinasi struktur perulangan dan struktur percabangan. Perulangan digunakan agar menu utama tetap ditampilkan selama pengguna belum memilih keluar. Sementara itu, percabangan digunakan untuk menentukan tindakan yang harus dijalankan berdasarkan input pengguna. Mekanisme ini memungkinkan aplikasi bekerja secara terstruktur dan menjaga konsistensi navigasi antar menu.

Dalam implementasi program, setiap topik utama ditempatkan dalam modul yang berbeda. File main.cpp berfungsi sebagai pusat kontrol program, yaitu menampilkan menu utama dan mengatur perpindahan antar submenu. File menu.h digunakan untuk mendukung tampilan antarmuka, seperti membersihkan layar, menampilkan jeda, dan menampilkan elemen-elemen menu. Selanjutnya, materi Array & Array Multi Dimensi ditempatkan pada modul tersendiri, demikian pula materi Pointer & Struct, Procedure & Fungsi, Sorting, dan Searching. Dengan rancangan tersebut, setiap modul memiliki tanggung jawab masing-masing sesuai topik materi yang dibahas.

Pada modul Array & Array Multi Dimensi, aplikasi dirancang untuk menampilkan materi seperti string, array statis, array dinamis, array satu dimensi, array dua dimensi, array

tiga dimensi, serta operasi matriks. Pada modul Pointer & Struct, aplikasi menampilkan materi pointer, struct, array of struct, pointer to struct, dan union. Pada modul Procedure & Fungsi, aplikasi menampilkan pembahasan mengenai procedure, fungsi, parameter, variabel lokal dan global, variabel static, serta fungsi rekursif. Adapun pada modul Sorting dan Searching, aplikasi menampilkan penjelasan teori, langkah algoritma, contoh implementasi, dan simulasi proses perhitungan secara bertahap. Dari sisi rancangan antarmuka, aplikasi dibuat sesederhana mungkin agar fokus utama tetap berada pada isi materi dan proses algoritma. Penggunaan tampilan teks dipilih karena sesuai dengan tujuan aplikasi sebagai media pembelajaran dasar yang menekankan logika dan struktur program. Dengan demikian, pengguna dapat lebih mudah memahami keterkaitan antara teori, implementasi kode, dan hasil proses algoritma yang ditampilkan oleh aplikasi.

Secara keseluruhan, perancangan program AlgoLearn menitikberatkan pada tiga aspek utama, yaitu kemudahan navigasi, keteraturan struktur modul, dan kejelasan penyajian materi. Ketiga aspek tersebut menjadi dasar dalam pengembangan aplikasi agar dapat berfungsi sebagai media pembelajaran yang sederhana namun tetap informatif dan mudah digunakan.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Array & Array Multi Dimensi

4.1.1 String

String adalah kumpulan karakter yang digunakan untuk menyimpan teks dalam sebuah variabel. Dalam bahasa pemrograman C atau C++, string biasanya disimpan dalam bentuk array karakter yang diakhiri dengan karakter null (\0). String sering digunakan untuk menyimpan data berupa teks seperti nama, alamat, atau kalimat.

Contoh program:

```
#include <iostream>
#include <conio.h>
using namespace std;

int main() {
    char kata[6];
    kata[0] = 'H';
    kata[1] = 'e';
    kata[2] = 'b';
    kata[3] = 'a';
    kata[4] = 't';
    kata[5] = '\0';

    cout << "Isi string adalah : " << kata << endl;
}
```

4.1.2 Array Statis

Array statis adalah array yang ukuran elemennya sudah ditentukan saat program ditulis dan tidak dapat diubah selama program dijalankan. Semua elemen array disimpan secara berurutan di dalam memori.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int angka[5] = {10, 20, 30, 40, 50};

    for(int i = 0; i < 5; i++) {
        cout << angka[i] << endl;
    }

    return 0;
}
```

4.1.3 Array Dinamis

Array dinamis adalah array yang ukuran elemennya dapat ditentukan saat program sedang berjalan (runtime). Dalam C++, array dinamis biasanya dibuat menggunakan operator new untuk mengalokasikan memori dan delete untuk menghapusnya setelah tidak digunakan.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Masukkan jumlah data: ";
    cin >> n;

    int *data = new int[n];

    for(int i = 0; i < n; i++) {
        cout << "Data ke-" << i+1 << ": ";
        cin >> data[i];
    }

    cout << "Data yang dimasukkan:\n";
    for(int i = 0; i < n; i++) {
        cout << data[i] << endl;
    }

    delete[] data;
}
```

4.1.4 Array 1 Dimensi

Array satu dimensi adalah struktur data yang digunakan untuk menyimpan beberapa nilai dengan tipe data yang sama dalam satu baris indeks. Setiap elemen dalam array dapat diakses menggunakan nomor indeks yang dimulai dari 0.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int nilai[4];

    for(int i = 0; i < 4; i++) {
        cout << "Masukkan nilai: ";
        cin >> nilai[i];
    }
```

```
    cout << "Daftar nilai:\n";
    for(int i = 0; i < 4; i++) {
        cout << nilai[i] << endl;
    }

    return 0;
}
```

4.1.5 Array Multi Dimensi (2 Dimensi)

Array dua dimensi adalah array yang memiliki dua indeks, yaitu baris dan kolom. Struktur ini sering digunakan untuk menyimpan data dalam bentuk tabel atau matriks.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int matriks[2][3] = {
        {1,2,3},
        {4,5,6}
    };

    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 3; j++) {
            cout << matriks[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

4.1.6 Array Multi Dimensi (3 Dimensi)

Array tiga dimensi merupakan pengembangan dari array dua dimensi dengan tambahan satu dimensi lagi. Struktur ini dapat dibayangkan sebagai kumpulan beberapa matriks yang disusun dalam satu variabel array.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int data[2][2][2] = {
        {{1,2},{3,4}},
        {{5,6},{7,8}}
```

```
    };

    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            for(int k=0;k<2;k++){
                cout << data[i][j][k] << " ";
            }
            cout << endl;
        }
        cout << endl;
    }

    return 0;
}
```

4.1.7 Penjumlahan Matriks

Penjumlahan matriks dilakukan dengan menjumlahkan elemen-elemen yang berada pada posisi yang sama di kedua matriks.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int A[2][2] = {{1,2},{3,4}};
    int B[2][2] = {{5,6},{7,8}};
    int C[2][2];

    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    cout<<"Hasil Penjumlahan Matriks:\n";
    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            cout<<C[i][j]<<" ";
        }
        cout<<endl;
    }
}
```

4.1.8 Pengurangan Matriks

Pengurangan matriks dilakukan dengan mengurangkan elemen-elemen yang berada pada posisi yang sama pada kedua matriks.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int A[2][2] = {{5,6},{7,8}};
    int B[2][2] = {{1,2},{3,4}};
    int C[2][2];

    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            C[i][j] = A[i][j] - B[i][j];
        }
    }

    cout<<"Hasil Pengurangan Matriks:\n";
    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            cout<<C[i][j]<<" ";
        }
        cout<<endl;
    }
}
```

4.1.9 Perkalian Matriks

Perkalian matriks dilakukan dengan mengalikan baris dari matriks pertama dengan kolom dari matriks kedua kemudian menjumlahkannya.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int A[2][2]={{1,2},{3,4}};
    int B[2][2]={{5,6},{7,8}};
    int C[2][2];

    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            C[i][j]=0;
            for(int k=0;k<2;k++){
                C[i][j]+=A[i][k]*B[k][j];
            }
        }
    }

    cout<<"Hasil Perkalian Matriks:\n";
    for(int i=0;i<2;i++){
```

```
        for(int j=0;j<2;j++){
            cout<<C[i][j]<<" ";
        }
        cout<<endl;
    }
}
```

4.1.10 Transpose Matriks

Transpose matriks adalah proses menukar posisi baris menjadi kolom.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int A[2][3]={{1,2,3},{4,5,6}};

    cout<<"Transpose Matriks:\n";

    for(int i=0;i<3;i++){
        for(int j=0;j<2;j++){
            cout<<A[j][i]<<" ";
        }
        cout<<endl;
    }
}
```

4.1.11 Invers Matriks

Invers matriks adalah matriks yang jika dikalikan dengan matriks asal akan menghasilkan matriks identitas. Invers biasanya hanya dapat dilakukan pada matriks persegi.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    float A[2][2]={{4,7},{2,6}};
    float det;

    det = (A[0][0]*A[1][1]) - (A[0][1]*A[1][0]);

    cout<<"Invers Matriks:\n";

    cout<<A[1][1]/det<<" "<<-A[0][1]/det<<endl;
    cout<<-A[1][0]/det<<" "<<A[0][0]/det<<endl;
}
```

4.2 Pointer & Struct

4.2.1 Pointer

Pointer adalah variabel yang digunakan untuk menyimpan alamat memori dari variabel lain. Dengan menggunakan pointer, program dapat mengakses atau memanipulasi data melalui alamat memori tersebut. Pointer biasanya digunakan dalam pengelolaan memori, pengiriman data ke fungsi, serta dalam struktur data seperti linked list dan tree.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int angka = 10;
    int *ptr;

    ptr = &angka;

    cout << "Nilai angka: " << angka << endl;
    cout << "Alamat angka: " << &angka << endl;
    cout << "Nilai pointer: " << *ptr << endl;

    return 0;
}
```

4.2.2 Struct

Struct (*structure*) adalah tipe data yang digunakan untuk menggabungkan beberapa variabel yang memiliki tipe data berbeda dalam satu kesatuan. Struct sering digunakan untuk merepresentasikan data yang lebih kompleks, seperti data mahasiswa, buku, atau produk.

Contoh program:

```
#include <iostream>
using namespace std;

struct Mahasiswa {
    string nama;
    int umur;
};

int main() {
    Mahasiswa mhs;

    mhs.nama = "Andi";
    mhs.umur = 20;

    cout << "Nama: " << mhs.nama << endl;
```

```
    cout << "Umur: " << mhs.umur << endl;

    return 0;
}
```

4.2.3 Array & Struct

Struct juga dapat digunakan dalam bentuk array untuk menyimpan banyak data dengan struktur yang sama. Dengan menggunakan array struct, program dapat menyimpan beberapa data yang memiliki tipe yang sama secara terorganisir.

Contoh program:

```
#include <iostream>
using namespace std;

struct Mahasiswa {
    string nama;
    int nilai;
};

int main() {
    Mahasiswa data[2];

    for(int i=0;i<2;i++){
        cout<<"Nama: ";
        cin>>data[i].nama;

        cout<<"Nilai: ";
        cin>>data[i].nilai;
    }

    for(int i=0;i<2;i++){
        cout<<data[i].nama<<" - "<<data[i].nilai<<endl;
    }
}
```

4.2.4 Pointer & Struct

Pointer juga dapat digunakan untuk mengakses atau memanipulasi anggota struct. Untuk mengakses anggota struct melalui pointer digunakan operator ->.

Contoh program:

```
#include <iostream>
using namespace std;

struct Buku {
    string judul;
    int harga;
};
```

```
int main() {
    Buku b1;
    Buku *ptr;

    ptr = &b1;

    ptr->judul = "Algoritma";
    ptr->harga = 50000;

    cout << ptr->judul << endl;
    cout << ptr->harga << endl;
}
```

4.2.5 Union

Union adalah tipe data yang memungkinkan beberapa variabel berbagi lokasi memori yang sama. Artinya, semua anggota union menggunakan ruang memori yang sama sehingga hanya satu nilai yang dapat disimpan pada satu waktu.

Contoh program:

```
#include <iostream>
using namespace std;

union Data {
    int angka;
    float nilai;
};

int main() {
    Data d;

    d.angka = 10;
    cout << "Angka: " << d.angka << endl;

    d.nilai = 3.14;
    cout << "Nilai: " << d.nilai << endl;
}
```

4.3 Prosedur & Fungsi

4.3.1 Prosedur

Prosedur adalah bagian dari program yang digunakan untuk menjalankan suatu tugas tertentu tetapi tidak mengembalikan nilai kepada program yang memanggilnya. Dalam bahasa pemrograman C++, prosedur biasanya ditulis menggunakan tipe data void. Penggunaan prosedur membantu membuat program menjadi lebih terstruktur dan mudah dipahami karena program yang panjang dapat dibagi menjadi beberapa bagian kecil yang memiliki fungsi masing-masing.

Contoh program:

```
#include <iostream>
using namespace std;

void tampilPesan(){
    cout << "Selamat Belajar Pemrograman!" << endl;
}

int main(){
    tampilPesan();
    return 0;
}
```

4.3.2 Fungsi

Fungsi adalah blok program yang digunakan untuk melakukan suatu proses tertentu dan mengembalikan nilai (*return value*) kepada program yang memanggilnya. Fungsi biasanya digunakan ketika hasil dari suatu proses ingin digunakan kembali dalam bagian lain dari program. Dengan menggunakan fungsi, program menjadi lebih efisien, terstruktur, dan mudah dipelihara.

Contoh program:

```
#include <iostream>
using namespace std;

int tambah(int a, int b){
    return a + b;
}

int main(){
    int hasil;

    hasil = tambah(5,3);

    cout << "Hasil penjumlahan: " << hasil << endl;

    return 0;
}
```

4.3.3 Parameter

Parameter adalah variabel yang digunakan untuk menerima nilai yang dikirimkan ketika suatu fungsi atau prosedur dipanggil. Parameter berfungsi sebagai perantara untuk mengirimkan data dari program utama ke dalam fungsi atau prosedur yang digunakan.

Parameter dibagi menjadi dua jenis yaitu:

1. Parameter Formal : Parameter yang dituliskan pada saat deklarasi fungsi atau prosedur.
2. Parameter Aktual : Nilai atau data yang diberikan saat fungsi atau prosedur dipanggil.

Contoh program:

```
#include <iostream>
using namespace std;

int kali(int x, int y){
    return x * y;
}

int main(){
    cout << kali(4,6);
}
```

4.3.4. Variabel Lokal

Variabel global adalah variabel yang dideklarasikan di luar fungsi utama maupun fungsi lainnya sehingga variabel tersebut dapat digunakan oleh semua fungsi yang ada di dalam program.

Contoh program:

```
#include <iostream>
using namespace std;

int nilai = 10;

void tampil(){
    cout << nilai << endl;
}

int main(){
    tampil();
}
```

4.3.5 Variabel Global

Variabel lokal adalah variabel yang dideklarasikan di dalam suatu fungsi sehingga hanya dapat digunakan pada fungsi tempat variabel tersebut dibuat. Variabel ini tidak dapat diakses oleh fungsi lain di dalam program.

Contoh program:

```
#include <iostream>
using namespace std;
```

```
void contoh(){
    int angka = 5;
    cout << angka;
}

int main(){
    contoh();
}
```

4.3.6 Variabel *Static*

Variabel *static* adalah variabel yang dideklarasikan dengan kata kunci *static* di dalam suatu fungsi. Variabel ini memiliki sifat khusus yaitu nilainya akan tetap tersimpan meskipun fungsi tersebut selesai dijalankan. Ketika fungsi dipanggil kembali, variabel *static* akan melanjutkan nilai terakhir yang telah disimpan sebelumnya.

Contoh program:

```
#include <iostream>
using namespace std;

void hitung(){
    static int nilai = 0;
    nilai++;
    cout << nilai << endl;
}

int main(){
    hitung();
    hitung();
    hitung();
}
```

4.3.4 Fungsi Rekursif

Fungsi rekursif adalah fungsi yang memanggil dirinya sendiri untuk menyelesaikan suatu permasalahan. Metode ini biasanya digunakan untuk menyelesaikan masalah yang memiliki pola berulang, seperti perhitungan faktorial, deret Fibonacci, atau algoritma tertentu.

Dalam penggunaan fungsi rekursif harus terdapat kondisi penghentian (base case) agar proses pemanggilan fungsi tidak berlangsung tanpa batas. Kondisi ini berfungsi untuk menghentikan proses rekursi ketika syarat tertentu telah terpenuhi.

Contoh program:

```
#include <iostream>
using namespace std;

int faktorial(int n){
    if(n == 1)
        return 1;
    else
        return n * faktorial(n-1);
}

int main(){
    cout << faktorial(5);
}
```

4.4 *Sorting*

Sorting adalah proses mengurutkan data berdasarkan urutan tertentu, misalnya dari nilai terkecil ke terbesar (*ascending*) atau terbesar ke terkecil (*descending*). Proses ini sering digunakan untuk mempermudah pencarian data dan pengolahan informasi.

4.4.1 Insertion Sort

Insertion Sort bekerja dengan cara menyisipkan elemen pada posisi yang benar di bagian array yang sudah terurut.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int data[5] = {7,4,5,2,9};

    for(int i=1;i<5;i++){
        int key = data[i];
        int j = i-1;

        while(j>=0 && data[j] > key){
            data[j+1] = data[j];
            j--;
        }

        data[j+1] = key;
    }

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

4.4.2 Heap Sort

Heap Sort merupakan algoritma pengurutan yang menggunakan konsep heap atau pohon biner untuk menyusun data sehingga proses pengurutan dapat dilakukan dengan lebih efisien.

Contoh program:

```
#include <iostream>
using namespace std;

void swap(int &a, int &b){
    int temp = a;
    a = b;
    b = temp;
}

void heapify(int arr[], int n, int i){
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if(left < n && arr[left] > arr[largest])
        largest = left;

    if(right < n && arr[right] > arr[largest])
        largest = right;

    if(largest != i){
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n){

    for(int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for(int i = n-1; i > 0; i--){
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main(){

    int data[5] = {4,10,3,5,1};

    heapSort(data,5);

    for(int i=0;i<5;i++)
```

```
        cout<<data[i]<<" ";
}
```

4.4.3 Merge Sort

Merge Sort menggunakan metode *divide and conquer* yaitu membagi array menjadi beberapa bagian kecil kemudian menggabungkannya kembali dalam kondisi terurut.

Contoh program:

```
#include <iostream>
using namespace std;

void merge(int arr[], int l, int m, int r){
    int i=l, j=m+1, k=l;
    int temp[100];

    while(i<=m && j<=r){
        if(arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while(i<=m)
        temp[k++] = arr[i++];

    while(j<=r)
        temp[k++] = arr[j++];

    for(int x=l;x<=r;x++)
        arr[x] = temp[x];
}

void mergesort(int arr[], int l, int r){
    if(l<r){
        int m = (l+r)/2;
        mergesort(arr,l,m);
        mergesort(arr,m+1,r);
        merge(arr,l,m,r);
    }
}

int main(){
    int data[5]={8,3,7,4,2};

    mergesort(data,0,4);

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
```

```
}
```

4.4.4 Quick Sort

Quick Sort adalah algoritma pengurutan yang menggunakan metode *divide and conquer*, yaitu dengan memilih satu elemen sebagai pivot, kemudian membagi elemen lain menjadi dua bagian: yang lebih kecil dari pivot dan yang lebih besar dari pivot.

Contoh program:

```
#include <iostream>
using namespace std;

void swap(int &a, int &b){
    int temp = a;
    a = b;
    b = temp;
}

int partition(int arr[], int low, int high){
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++){
        if(arr[j] < pivot){
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i+1], arr[high]);
    return i+1;
}

void quickSort(int arr[], int low, int high){
    if(low < high){
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi-1);
        quickSort(arr, pi+1, high);
    }
}

int main(){
    int data[5] = {9, 4, 6, 2, 8};

    quickSort(data,0,4);

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

4.4.5 Tree Sort

Tree Sort adalah algoritma pengurutan yang menggunakan struktur Binary Search Tree (BST). Data dimasukkan ke dalam tree, kemudian dilakukan inorder traversal untuk mendapatkan data yang sudah terurut.

Contoh program:

```
#include <iostream>
using namespace std;

struct Node{
    int data;
    Node* left;
    Node* right;
};

Node* newNode(int data){
    Node* node = new Node();
    node->data = data;
    node->left = node->right = NULL;
    return node;
}

Node* insert(Node* root, int data){
    if(root == NULL)
        return newNode(data);

    if(data < root->data)
        root->left = insert(root->left,data);
    else
        root->right = insert(root->right,data);

    return root;
}

void inorder(Node* root){
    if(root != NULL){
        inorder(root->left);
        cout<<root->data<<" ";
        inorder(root->right);
    }
}

int main(){
    int data[5] = {7,3,9,1,5};
    Node* root = NULL;

    for(int i=0;i<5;i++)
        root = insert(root,data[i]);

    inorder(root);
```

```
}
```

4.4.6 Exchange Sort

Exchange Sort adalah metode pengurutan yang dilakukan dengan membandingkan dua elemen dan menukarnya jika urutannya salah.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int data[5] = {8,3,6,2,7};

    for(int i=0;i<4;i++){
        for(int j=i+1;j<5;j++){
            if(data[i] > data[j]){
                int temp = data[i];
                data[i] = data[j];
                data[j] = temp;
            }
        }
    }

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

4.4.7 Radix Sort

Radix Sort adalah algoritma pengurutan yang bekerja dengan mengurutkan angka berdasarkan digitnya, mulai dari digit paling kecil hingga digit paling besar.

Contoh program:

```
#include <iostream>
using namespace std;

int getMax(int arr[], int n){
    int mx = arr[0];
    for(int i=1;i<n;i++)
        if(arr[i] > mx)
            mx = arr[i];
    return mx;
}

void countSort(int arr[], int n, int exp){
    int output[5];
    int count[10] = {0};
```

```
    for(int i=0;i<n;i++)
        count[(arr[i]/exp)%10]++;

    for(int i=1;i<10;i++)
        count[i] += count[i-1];

    for(int i=n-1;i>=0;i--){
        output[count[(arr[i]/exp)%10]-1] = arr[i];
        count[(arr[i]/exp)%10]--;
    }

    for(int i=0;i<n;i++)
        arr[i] = output[i];
}

void radixSort(int arr[], int n){
    int m = getMax(arr,n);

    for(int exp=1; m/exp>0; exp*=10)
        countSort(arr,n,exp);
}

int main(){
    int data[5]={170,45,75,90,802};

    radixSort(data,5);

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

4.4.8 Shell Sort

Shell Sort merupakan pengembangan dari Insertion Sort yang melakukan pengurutan dengan membandingkan elemen yang memiliki jarak tertentu (*gap*).

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int data[6] = {12,34,54,2,3,8};
    int n = 6;

    for(int gap = n/2; gap > 0; gap /= 2){
        for(int i = gap; i < n; i++){
            int temp = data[i];
            int j;

            for(j=i; j>=gap && data[j-gap] > temp; j-=gap)
```

```
                data[j] = data[j-gap];

            data[j] = temp;
        }
    }

    for(int i=0;i<n;i++)
        cout<<data[i]<<" ";
}
```

4.4.9 Bubble Sort

Bubble Sort adalah metode pengurutan yang bekerja dengan cara membandingkan dua elemen yang berdekatan dan menukarnya jika urutannya salah.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int data[5]={5,1,4,2,8};

    for(int i=0;i<4;i++){
        for(int j=0;j<4-i;j++){
            if(data[j] > data[j+1]){
                int temp = data[j];
                data[j] = data[j+1];
                data[j+1] = temp;
            }
        }
    }

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

4.4.10 Selection Sort

Selection Sort adalah metode pengurutan yang bekerja dengan cara mencari elemen terkecil dari array kemudian menukarnya dengan posisi pertama secara berulang.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int data[5] = {29,10,14,37,13};
```

```
    for(int i=0;i<4;i++){
        int min = i;

        for(int j=i+1;j<5;j++){
            if(data[j] < data[min])
                min = j;
        }

        int temp = data[i];
        data[i] = data[min];
        data[min] = temp;
    }

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

4.5 Searching

Searching adalah proses dalam pemrograman yang digunakan untuk menemukan suatu data tertentu di dalam kumpulan data seperti array atau daftar nilai. Proses ini dilakukan dengan membandingkan data yang dicari dengan setiap elemen yang ada sampai data tersebut ditemukan atau dinyatakan tidak ada. Dalam algoritma dan pemrograman, terdapat beberapa metode *searching* yang sering digunakan, seperti Sequential Search, Binary Search, Fibonacci Search, dan Interpolation Search, yang masing-masing memiliki cara kerja dan tingkat efisiensi yang berbeda.

4.5.1 Fibonacci Search

Fibonacci Search adalah metode pencarian pada array yang sudah terurut dengan menggunakan bilangan Fibonacci untuk menentukan posisi pembagian pencarian. Metode ini mirip dengan Binary Search tetapi menggunakan perbandingan jarak berdasarkan deret Fibonacci.

Contoh program:

```
#include <iostream>
#include <algorithm>
using namespace std;

int fibonacciSearch(int arr[], int n, int x)
{
    int fib2 = 0;      // (m-2)'th Fibonacci
    int fib1 = 1;      // (m-1)'th Fibonacci
    int fib = fib2 + fib1; // m'th Fibonacci

    while (fib < n) {
        fib2 = fib1;
        fib1 = fib;
        fib = fib2 + fib1;
```

```
    }

    int offset = -1;

    while (fib > 1) {
        int i = min(offset + fib2, n - 1);

        if (arr[i] < x) {
            fib = fib1;
            fib1 = fib2;
            fib2 = fib - fib1;
            offset = i;
        }
        else if (arr[i] > x) {
            fib = fib2;
            fib1 = fib1 - fib2;
            fib2 = fib - fib1;
        }
        else
            return i;
    }

    if (fib1 && arr[offset + 1] == x)
        return offset + 1;

    return -1;
}

int main()
{
    int data[] = {5, 10, 15, 20, 25, 30, 35};
    int n = 7;
    int cari = 25;

    int hasil = fibonacciSearch(data, n, cari);

    if (hasil != -1)
        cout << "Data ditemukan pada indeks ke-" << hasil;
    else
        cout << "Data tidak ditemukan";

    return 0;
}
```

4.5.2 Binary Search

Binary Search adalah metode pencarian data pada array yang sudah terurut dengan cara membagi data menjadi dua bagian secara berulang sampai data yang dicari ditemukan atau tidak ada lagi bagian yang bisa diperiksa. Algoritma ini lebih cepat

dibandingkan pencarian berurutan karena tidak perlu memeriksa semua elemen satu per satu.

Contoh program:

```
#include <iostream>
using namespace std;

int main()
{
    int data[8] = {2, 5, 8, 12, 16, 23, 38, 56};
    int kiri = 0;
    int kanan = 7;
    int tengah;
    int cari;
    bool ditemukan = false;

    cout << "Masukkan angka yang dicari: ";
    cin >> cari;

    while (kiri <= kanan)
    {
        tengah = (kiri + kanan) / 2;

        if (data[tengah] == cari)
        {
            ditemukan = true;
            cout << "Data ditemukan pada indeks ke-" << tengah << endl;
            break;
        }
        else if (cari > data[tengah])
        {
            kiri = tengah + 1;
        }
        else
        {
            kanan = tengah - 1;
        }
    }

    if (!ditemukan)
    {
        cout << "Data tidak ditemukan dalam array." << endl;
    }

    return 0;
}
```

4.5.3 Interpolation Search

Interpolation Search adalah metode pencarian pada array yang sudah terurut dengan cara memperkirakan posisi data yang dicari berdasarkan nilai data tersebut.

Algoritma ini bekerja lebih cepat jika data dalam array memiliki distribusi nilai yang relatif merata. Berbeda dengan Binary Search yang selalu mengambil elemen tengah, Interpolation Search menghitung posisi kemungkinan dari nilai yang dicari menggunakan rumus tertentu.

Contoh program:

```
#include <iostream>
using namespace std;

int interpolationSearch(int arr[], int n, int x)
{
    int low = 0;
    int high = n - 1;

    while (low <= high && x >= arr[low] && x <= arr[high])
    {
        int pos = low + ((double)(high - low) / (arr[high] - arr[low])) * (x -
arr[low]);

        if (arr[pos] == x)
            return pos;

        if (arr[pos] < x)
            low = pos + 1;
        else
            high = pos - 1;
    }

    return -1;
}

int main()
{
    int data[] = {10, 20, 30, 40, 50, 60, 70};
    int n = 7;
    int cari = 50;

    int hasil = interpolationSearch(data, n, cari);

    if (hasil != -1)
        cout << "Data ditemukan pada indeks ke-" << hasil;
    else
        cout << "Data tidak ditemukan";

    return 0;
}
```

4.5.4 Sequential Search

Sequential Search adalah metode pencarian data yang dilakukan dengan cara memeriksa setiap elemen dalam array secara berurutan mulai dari elemen pertama hingga elemen terakhir sampai data yang dicari ditemukan atau seluruh data telah diperiksa.

Metode ini merupakan algoritma pencarian yang paling sederhana karena tidak memerlukan data dalam keadaan terurut.

Contoh program:

```
#include <iostream>
using namespace std;

int main()
{
    int data[6] = {14, 7, 25, 30, 18, 9};
    int cari;
    int posisi = -1;

    cout << "Masukkan nilai yang ingin dicari: ";
    cin >> cari;

    for(int i = 0; i < 6; i++)
    {
        if(data[i] == cari)
        {
            posisi = i;
            break;
        }
    }

    if(posisi != -1)
    {
        cout << "Data ditemukan pada indeks ke-" << posisi << endl;
    }
    else
    {
        cout << "Data tidak ditemukan dalam array." << endl;
    }

    return 0;
}
```

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Algoritma dan pemrograman merupakan materi yang penting dalam pembelajaran ilmu komputer karena berperan dalam melatih kemampuan berpikir logis, problem solving, dan penyusunan solusi program yang efisien. Namun, materi ini sering dianggap sulit oleh mahasiswa karena konsep-konsep yang dipelajari bersifat abstrak, dinamis, dan cukup kompleks. Oleh karena itu, diperlukan media pembelajaran yang dapat membantu mahasiswa memahami materi secara lebih terstruktur dan aplikatif.

AlgoLearn merupakan aplikasi pembelajaran berbasis C++ dengan pendekatan *Command Line Interface* (CLI) yang dirancang untuk membantu pengguna mempelajari materi algoritma pemrograman dan struktur data. Aplikasi ini memuat beberapa topik utama, yaitu Array & Array Multi Dimensi, Pointer & Struct, Prosedur & Fungsi, *Sorting*, dan *Searching*. Melalui penyajian materi yang terstruktur, contoh implementasi program, dan simulasi sederhana pada beberapa algoritma, aplikasi ini dapat menjadi media pembelajaran alternatif untuk membantu memahami konsep dasar algoritma dan struktur data.

Berdasarkan hasil perancangan dan implementasi, aplikasi AlgoLearn telah dapat digunakan sebagai media pembelajaran dasar untuk materi algoritma pemrograman dan struktur data. Aplikasi ini menyajikan materi dalam bentuk menu yang terstruktur, disertai contoh program dan simulasi sederhana pada beberapa topik, sehingga pengguna dapat memahami hubungan antara konsep teori dan implementasi program secara lebih jelas. Dengan demikian, AlgoLearn dapat dimanfaatkan sebagai sarana bantu belajar yang mendukung pemahaman konsep dasar algoritma pemrograman dan struktur data.

5.2 Saran

Untuk pengembangan program lebih lanjut, dapat dilakukan penambahan fitur visual yang lebih interaktif, sehingga proses algoritma dapat dipahami dengan lebih jelas dan menarik. Pengembangan antarmuka ke bentuk *Graphical User Interface* (GUI) atau berbasis web dapat dipertimbangkan agar aplikasi lebih mudah digunakan dan memiliki tampilan yang lebih modern.

DAFTAR PUSTAKA

Andreev, N. A., et al. (2021). The Effectiveness Of Using Visualization Tools And Forms In Distance Learning. *Revista Tempos e Espaços em Educação*, 14(33).

Ginting, N. H. S. (2023). *Pengantar Struktur Data*. Deli Serdang: Mifandi Mandiri Digital.

Hanief, S., & Jepriana, W. I. (2020). *Konsep Algoritme dan Aplikasinya dalam Bahasa Pemrograman C++*. Yogyakarta: Penerbit ANDI.

Khesya, N. (2021). Mengenal flowchart dan pseudocode dalam algoritma dan pemrograman. *Preprints*, 1, 1–15. Diakses pada 28 Maret 2026, dari (https://osf.io/dq45e](https://osf.io/dq45e).

Kitchin, R. (2017). Thinking Critically About And Researching Algorithms. *Information, Communication & Society*, 20(1), 14–29.

Kusnadi, Y., et al. (2025*). Algoritma dan Pemrograman Dasar*. Jakarta: Bukuloka Literasi Bangsa.

Lestanto, Y. (2023, 31 Juli). Modul panduan struktur data menggunakan MATLAB. Diakses pada 28 Maret 2026, dari (https://repository.bakrie.ac.id/7818/).

Mtaho, B. A., & Mselle, J. L. (2024). Difficulties In Learning The Data Structures Course: Literature Review. *The Journal of Informatics*, 4(1), 26–55.

Perhac, P., & Simonak, S. (2022). Interactive System For Algorithm And Data Structure Visualization. *Computer Science Journal of Moldova*, 30(1), 28–48.

Pratama, M. A. (2020). Struktur data array dua dimensi pada pemrograman C++. Preprints, 1, 1–8. Diakses pada 28 Maret 2026, dari (https://doi.org/10.31219/osf.io/vyech).

Pujiono, P. I. (2025). Algoritma Counting Sort Vs Algoritma Pengurutan Modern: Analisis Efisiensi Memori Dan Waktu Komputasi. *Jurnal Informatika dan Teknik Elektro Terapan*, 13(3), 135–142.

Ritonga, A., & Yahfizham. (2023). Studi Literatur Perbandingan Bahasa Pemrograman C++ Dan Bahasa Pemrograman Python Pada Algoritma Pemrograman. *Jurnal Teknik Informatika dan Teknologi Informasi*, 3(3), 56–63.

Talumewo, M. D., et al. (2025). *Algoritma dan Struktur Data*. Agam: Yayasan Tri Edukasi Ilmiah.